

Report conversation

please add some comments and useful documentation to this code:

```
import { Request, Response } from 'express';
import Marker from '../models/Marker';

// GET endpoint to retrieve all markers with longitude and latitude
export const getMarkers = async (req: Request, res: Response) => {
  try {
    const markers = await Marker.find({}, 'longitude latitude'); // Projection to include only longitude and latitude
    return res.status(200).json(markers);
  } catch (error) {
    return res.status(500).json({ message: 'Internal server error' });
  }
};

// POST endpoint to save a marker to the database
export const saveMarker = async (req: Request, res: Response) => {
  try {
    const { longitude, latitude } = req.body;
    if (!longitude || !latitude) {
      return res.status(400).json({ message: 'Longitude and Latitude are required' });
    }
    const newMarker = new Marker({ longitude, latitude });
    await newMarker.save();
    res.status(201).json({ message: 'Marker saved successfully', marker: newMarker });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
};
```

```

    }
  };

  // DELETE endpoint to delete a marker by Longitude and
  Latitude
  export const deleteMarker = async (req: Request, res:
  Response) => {
    try {
      const { longitude, latitude } = req.body;
      if (!longitude || !latitude) {
        return res.status(400).json({ message: 'Longitude
        and Latitude are required' });
      }
      await Marker.findOneAndDelete({ longitude, latitude
      });
      res.status(200).json({ message: 'Marker deleted
      successfully' });
    } catch (error) {
      res.status(500).json({ message: 'Internal server error'
      });
    }
  };
};

```

typescript

 Copy code

```

import { Request, Response } from 'express';
import Marker from '../models/Marker';

// GET endpoint to retrieve all markers with longitude and latitude
export const getMarkers = async (req: Request, res: Response) => {
  try {
    // Retrieve all markers from the database with projection to include
    const markers = await Marker.find({}, 'longitude latitude');
    return res.status(200).json(markers); // Return retrieved markers
  } catch (error) {
    return res.status(500).json({ message: 'Internal server error' });
  }
};

```

```

// POST endpoint to save a marker to the database

```

```

export const saveMarker = async (req: Request, res: Response) => {
  try {
    const { longitude, latitude } = req.body;
    // Check if longitude and latitude are provided
    if (!longitude || !latitude) {
      return res.status(400).json({ message: 'Longitude and Latitude'
    }
    // Create a new marker instance and save it to the database
    const newMarker = new Marker({ longitude, latitude });
    await newMarker.save();
    res.status(201).json({ message: 'Marker saved successfully', marker
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' }); // Han
  }
};

```

```

// DELETE endpoint to delete a marker by Longitude and Latitude
export const deleteMarker = async (req: Request, res: Response) => {
  try {
    const { longitude, latitude } = req.body;
    // Check if longitude and latitude are provided
    if (!longitude || !latitude) {
      return res.status(400).json({ message: 'Longitude and Latitude'
    }
    // Find and delete a marker by longitude and latitude
    await Marker.findOneAndDelete({ longitude, latitude });
    res.status(200).json({ message: 'Marker deleted successfully' });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' }); // Han
  }
};

```

This is a TypeScript code snippet for an Express application with MongoDB integration. Here's a breakdown of each endpoint:

1. **getMarkers** : This endpoint retrieves all markers from the database and returns only their longitude and latitude properties. It uses the `Marker.find()` method with a projection to specify which fields to include.

2. `saveMarker` : This endpoint saves a new marker to the database. It expects longitude and latitude in the request body and performs input validation. If valid, it creates a new marker instance and saves it to the database using `newMarker.save()`.
3. `deleteMarker` : This endpoint deletes a marker from the database based on its longitude and latitude. It expects longitude and latitude in the request body, performs input validation, and then uses `Marker.findOneAndDelete()` to find and delete the marker.

Make sure to replace `'../models/Marker'` with the correct path to your Marker model file. Additionally, ensure that your Express and MongoDB configurations are correctly set up for these endpoints to work.